Project: ClueHunt — AI Powered Interactive Mystery Investigation Platform

---

1. Executive Summary

ClueHunt is a full-stack AI-powered interactive mystery game that uses Large Language Models (LLMs) to dynamically generate detective-style investigation scenarios.

The platform allows players to act as detectives by analyzing crime scenes, reviewing suspect statements, identifying contradictions, and discovering the single suspect who is lying.

The system combines a modern full-stack architecture with AI agent workflows to create scalable and replayable mystery experiences.

Problem Explored

Traditional mystery games rely on manually written cases, fixed storylines, and predefined clues. This limits replayability and requires continuous manual content creation.

The project explores whether AI can generate unique mystery cases while maintaining logical consistency and fair gameplay.

What Was Built

A complete AI-driven application consisting of:

- React-based interactive investigation interface.
- FastAPI backend API layer.
- LangGraph multi-agent AI workflow.
- LLM-powered case generation.
- AI validation system.
- Automated detective-style verdict explanation.

Outcome

The application successfully generates:

- Mystery scenarios.
- Crime scenes.
- Suspect profiles.
- Evidence chains.
- Interrogation statements.
- AI-generated verdict explanations.

Key Recommendation

LLM applications involving reasoning tasks should use validation workflows instead of directly exposing generated outputs. A generator + validator architecture improves reliability and reduces hallucinations.

---

2. Problem Statement & Objective

Problem Statement

Generating high-quality mystery content manually requires significant effort from writers and designers.

Additionally, AI-generated stories often suffer from:

- Contradicting information.
- Multiple possible answers.
- Weak evidence chains.
- Obvious clues.

The objective of this project was to evaluate how AI agents can generate structured mystery scenarios while ensuring logical correctness.

Objectives

The project aimed to:

- Build an AI-powered mystery generation system.
- Generate unique cases dynamically.
- Maintain exactly one valid solution.
- Validate AI-generated reasoning.

- Provide interactive detective experiences.

Technical Challenge

The main challenge was not only generating creative content but ensuring:

- The mystery remains solvable.
- Only one suspect is lying.
- Evidence logically connects to the final answer.

---

3. Research Summary

The project researched modern Generative AI application patterns, especially around LLM agents and workflow orchestration.

LLM-Based Content Generation

Large Language Models were evaluated for:

- Story creation.
- Character generation.
- Dialogue creation.
- Reasoning explanation.

However, direct generation introduces risks:

- Hallucinated facts.
- Inconsistent timelines.
- Invalid reasoning.

This influenced the decision to introduce AI validation stages.

Agent-Based Architecture Research

The project explored workflow-based AI systems where multiple AI stages handle different responsibilities.

The selected pattern:

Generator Agent
|
|
Validator Agent
|
|
Final Output

This provides better control compared to a single LLM call.

LangGraph Research

LangGraph was selected because it supports:

- Stateful workflows.
- Multiple AI nodes.
- Conditional execution.
- Shared state handling.

It allowed the application to implement:

- Case generation workflow.
- Validation workflow.
- Explanation workflow.

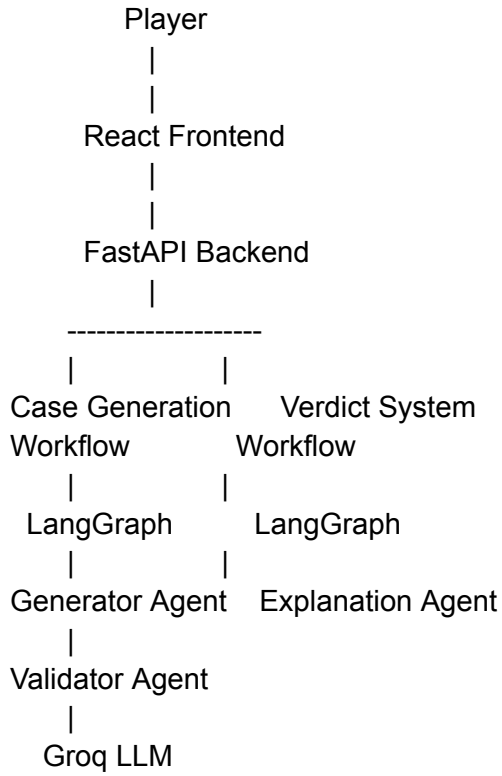
Key Research Findings

- LLMs are strong at creative generation.
- Validation agents improve reliability.
- Structured outputs reduce AI unpredictability.
- Agent workflows are suitable for complex AI applications.

---

4. Solution Architecture

High-Level Architecture



System Components

Frontend Layer

Built using:

- React
- Vite
- Tailwind CSS
- Lucide Icons

Responsibilities:

- Display investigation interface.

- Show scenes and suspects.
- Allow accusation submission.
- Display AI explanations.

Backend Layer

Built using:

- Python
- FastAPI

Responsibilities:

- API handling.
- AI workflow execution.
- Case management.
- Database communication.

AI Workflow Architecture

Phase 1: Case Generation

When a player starts a game:

User Request
|
|
Case Generator Agent
|
|
LLM Creates Mystery
|
|
Schema Validation
|
|
Logic Validator
|

|
Final Case

Generator Agent

The generator creates:

- Mystery scenario.
- Three suspects.
- Crime scenes.
- Evidence.
- Statements.

The prompt design prevents obvious clues by restricting giveaway words and forcing subtle reasoning.

Logic Validator Agent

The validator checks:

- Exactly one false statement exists.
- Evidence supports the final answer.
- No logical contradictions exist.

Phase 2: Player Investigation

The frontend presents:

Scenes

Contains:

- Locations.
- Environmental clues.
- Evidence.

Suspects

Contains:

- Background.
- Motives.
- Alibis.
- Statements.

The player compares evidence and identifies inconsistencies.

Phase 3: Verdict Explanation

After accusation:

Player Choice
|
|
Explanation Agent
|
|
Evidence Analysis
|
|
Detective Explanation

The explanation agent provides:

- Correctness result.
- Evidence breakdown.
- Reasoning chain.

---

5. Technology Stack

Technology	Purpose
React	Frontend UI

Vite	Frontend build system
Tailwind CSS	Custom styling
Lucide React	UI icons
Python	Backend development
FastAPI	REST API framework
LangGraph	AI workflow orchestration
LangChain Core	LLM integration
Groq API	LLM provider
Llama 3.1 70B	Mystery generation model
Pydantic	Data validation
MongoDB	Case persistence
Motor	Async MongoDB driver

---

6. Implementation Overview

Backend Modules

main.py

Handles:

- API routes.
- Application startup.
- Middleware.

---

ai_workflow.py

Contains:

- LangGraph workflows.
- Generator agent.

- Validator agent.
- Explanation agent.

---

schemas.py

Defines:

- Mystery case schema.
- Suspect schema.
- Evidence schema.
- API response models.

---

db.py

Provides:

- MongoDB storage.
- In-memory fallback storage.

Frontend Modules

CaseSetup

Responsible for:

- Starting investigations.
- Selecting difficulty.

InvestigationBoard

Displays:

- Crime scenes.
- Suspect information.
- Evidence.

AccusationPanel

Handles:

- Final suspect selection.

VerdictModal

Displays:

- AI detective explanation.

API Implementation

Health Check

GET /health

Provides:

- Server status.
- Active AI provider.
- Database mode.

Generate Case

POST /api/cases

Triggers:

- Case generation workflow.

Returns:

- MysteryCase object.

Retrieve Case

GET /api/cases/{case_id}

Fetches stored investigation.

Accusation

POST /api/accuse

Runs:

- Explanation workflow.

Returns:

- Verdict.
- Reasoning.

---

7. Challenges & Observations

1. LLM Hallucinations

Challenge

AI-generated mysteries may contain:

- Incorrect evidence.
- Conflicting statements.
- Multiple suspects appearing guilty.

Solution

Implemented:

- Validator agent.
- Structured schemas.
- Evidence consistency checks.

---

2. Maintaining Mystery Fairness

Challenge

Avoiding obvious answers.

Solution

Prompt engineering was used to:

- Remove giveaway clues.
- Encourage subtle evidence.
- Maintain balanced difficulty.

---

3. External AI Dependency

Challenge

LLM APIs may fail due to:

- Rate limits.
- Missing keys.
- Network issues.

Solution

Added:

- Deterministic fallback mystery generation.

---

8. Results & Evaluation

Functional Testing

Feature	Result
Dynamic case generation	Successful
AI validation	Successful
Investigation flow	Successful
Accusation system	Successful
AI explanation	Successful

Performance Observations

- Groq LLM provides low-latency generation.
- LangGraph provides predictable workflow execution.
- Validation improves output quality.

Failure Cases

Potential issues:

- Extremely complex mysteries.
- Long reasoning chains.
- Ambiguous AI outputs.

---

9. Limitations & Future Improvements

Current Limitations

- Limited player memory.
- No real-time suspect conversations.
- Difficulty balancing depends on generated cases.
- AI quality depends on model capability.

Future Improvements

Multi-Agent Investigation

Add specialized agents:

- Crime Scene Agent.
- Suspect Agent.
- Evidence Agent.
- Judge Agent.

Interactive Interrogation

Allow players to:

- Ask suspects questions.
- Receive dynamic answers.
- Challenge statements.

Adaptive Difficulty

System can analyze:

- Player performance.
- Solving speed.

Then adjust:

- Clue complexity.

- Number of suspects.
- Evidence difficulty.

Production Scaling

Future improvements:

- Cloud deployment.
- Distributed storage.
- User accounts.
- Multiplayer investigations.

---

10. Key Learnings & Recommendations

Learnings

- AI generation requires validation.
- Agent workflows improve control.
- Structured outputs reduce failures.
- LLMs work best when combined with traditional software engineering.

What Worked Well

- LangGraph simplified multi-step reasoning.
- FastAPI created clean backend separation.
- Pydantic improved reliability.
- Fallback systems improved resilience.

Recommended Architecture

For future AI applications:

Input

|
AI Agent
|
Validator
|
Processor
|
Final Output

Avoid direct:

Input → LLM → Output

without verification.

---

11. Deliverables

Completed:

- Full-stack ClueHunt application.
- AI workflow implementation.
- React UI.
- FastAPI backend.
- API documentation.
- Project documentation.

Repository Structure

ClueHunt/

backend/
├── app/
│ ├── ai_workflow.py
│ ├── db.py
│ ├── main.py
│ ├── schemas.py
│ └── settings.py

```
frontend/
├── src/
│   ├── components/
│   ├── App.jsx
│   ├── api.js
│   └── styles.css
```

Final Conclusion

ClueHunt demonstrates how modern Generative AI can be integrated into a reliable full-stack application.

By combining LLM generation with LangGraph-based validation workflows, the system achieves creative content generation while maintaining logical consistency.

The architecture can be extended beyond games into:

- AI simulations.
- Training platforms.
- Interactive storytelling.
- Investigation assistants.